



COURS ADMINISTRATION AVANCÉ DE BASES DE DONNÉES L3

IUT INFORMATIQUE BELFORT

CONCEPTS À ABORDER

- Comprendre la problématique de cohérences, de qualité et de sécurité des bases de données.
 - Gestion des espaces de données
 - Gestion des utilisateurs
 - Haute disponibilité (Clusterisation, réplication)
 - Sauvegarde / Restauration
 - Internationalisation
 - Fédération des bases de données (Communication entre les BDD)
 - Savoirs de référence à approfondir.

LA GESTION DES ESPACES DE DONNÉES

- Bonnes pratiques de l'administration des bases de données

Vous devez mettre en place un certain nombre de bonnes pratiques de gestion des données pour :

- Maintenir une haute qualité des données
- Assurer les performances de la base de données selon les objectifs fixés.

Dans un passé pas très lointain, les administrateurs avaient un rôle plus pratique

- ils n'avaient pas le support de l'IA et des processus automatisés d'aujourd'hui sur lequel s'appuyer.

En effet, cette réalité réduisait la quantité de données qu'un DBA pouvait gérer et rendait le travail de l'administrateur beaucoup moins stratégique qu'il ne l'est généralement maintenant.

Données à grande échelle: Les entreprises génèrent de plus en plus de données, ce qui complexifie d'avantage le travail de l'administrateur et conduit à de nouveaux métiers autour de la donnée (Big Data, Data science etc)

Optimisation: En optimisant le stockage des données, vous optimisez les performances d'accès à ces données par conséquent vous optimisez l'utilisation des ressources, la fiabilité, la sécurité, et plus encore.

LA GESTION DES ESPACES DE DONNÉES

- Bonnes pratiques de l'administration des bases de données

Conseils pour une gestion moderne des bases de données

- Établissez des objectifs à atteindre (en terme de performance, de disponibilité et de qualité)
Une stratégie de gestion de base de données concrète et ciblée doit refléter les besoins de l'entreprise
Définir les indicateurs que vous utiliserez pour suivre le bon fonctionnement de votre BDD.
- Établir des objectifs pertinents vous donne un cap à suivre pour ne pas vous égarer.
- Automatisation et amélioration des processus:
Vous pouvez également utiliser les données pour vous aider à remodeler vos processus, mettre en œuvre l'automatisation et apporter des ajustements.
- Établissez des politiques et procédures de sauvegarde et de restauration

LA GESTION DES ESPACES DE DONNÉES

- Bonnes pratiques de l'administration des bases de données
- Collecte et organisation des données: tout administrateur doit être formé sur vos procédures et politiques de collecte de données. L'accroissement de l'automatisation dans les différents processus peut conduire à la perte de connaissance. Assurez-vous que tout le monde est sur la même longueur d'onde sur la manière dont les logiciels développés contribuent à maintenir vos environnements.
- Protéger l'intégrité des données : Une corruption de votre base de données peut être dévastatrice, alors élaborer un plan pour les minimiser et permettez à votre équipe de les trouver et de les corriger lorsqu'elles se produisent (Monitoring).

Encrypter les données

- Surveillez vos données : De manière régulière, vérifiez vos bases de données pour leur exactitude et une éventuelle corruption des données.
- Établissez des points de repère : En tant qu'administrateur vous aiderez à établir des alertes pour protéger la base de données en signalant les problèmes lorsqu'ils se produisent. Vous devez connaître les exigences de vos clients (RPO et RTO) et être prête à agir en conséquence si des changements se produisent.
- Cartographiez vos processus: Pouvoir visualiser comment fonctionne votre base de données et voir l'ensemble du processus, de la collecte de données au traitement.

LA GESTION DES ESPACES DE DONNÉES

- L'importance de l'optimisation du stockage
 - Le coût du stockage
 - Les risques de corruption de vos données → Risques de perte pour l'entreprise
 - Les performances de la base de données.
 - Les données utiles et non utiles → Archivage

GESTION DES ESPACES DE DONNÉES DANS POSTGRESQL

- **Qu'est-ce qu'un tablespace ?**

Un tablespace est un emplacement sur le système de fichiers utilisé par PostgreSQL

pour stocker des données. Cela permet :

- De répartir les fichiers de base sur plusieurs disques
- De mieux gérer les performances et l'espace disque

- **Créer un tablespace**

- `CREATE TABLESPACE mon_espace OWNER postgres LOCATION '/chemin/vers/le/dossier';`

- **Utiliser un tablespace**

- **Créer une base dans un tablespace :**

- `CREATE DATABASE ma_base`
- `TABLESPACE mon_espace;`

Créer une table dans un tablespace spécifique :

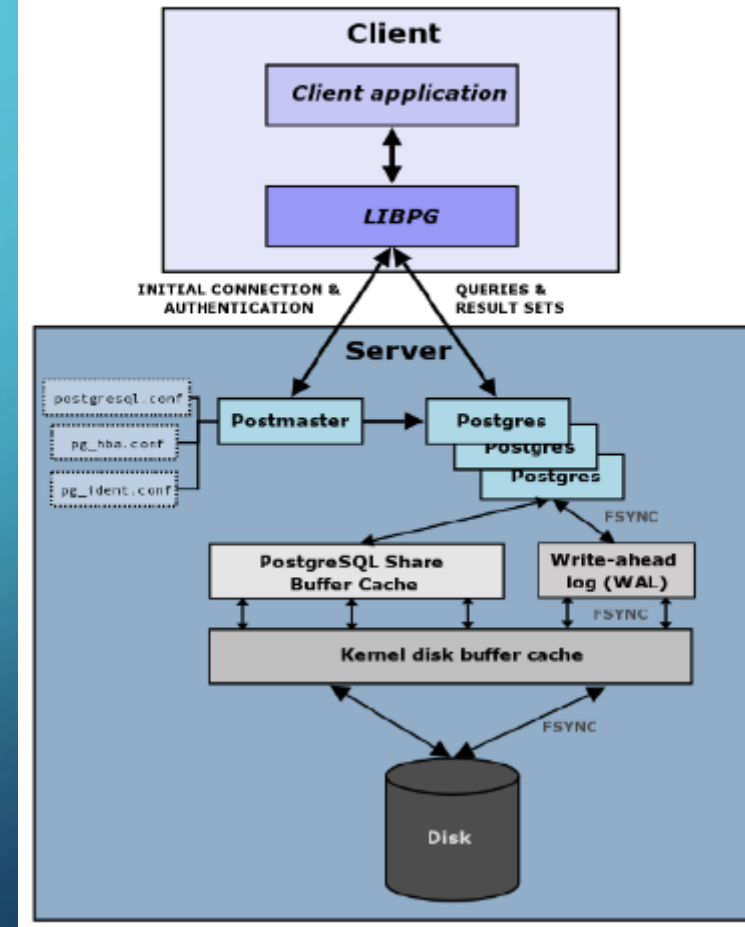
sql

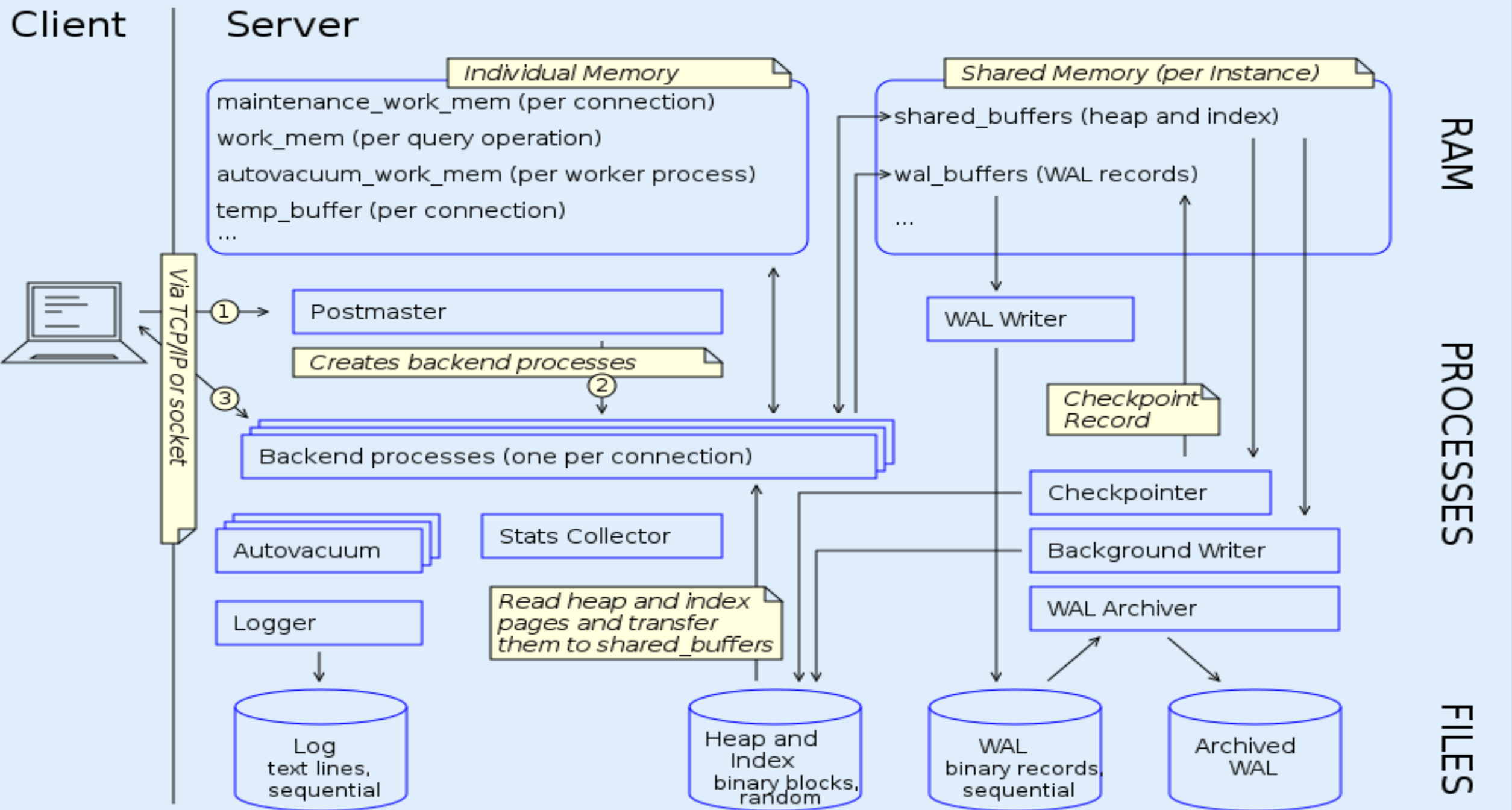
```
CREATE TABLE clients ( id SERIAL PRIMARY KEY, nom TEXT ) TABLESPACE mon_espace;
```

ARCHITECTURE

- L'importance de l'optimisation du stockage
- Postmaster :
processus de supervision
Gère les demandes de connexion
Rattaché à shmmem
- Utility processes :
Bgwriter, Syslogger, Archiver, Statscollector, Autovacuum launcher,
WAL sender/receiver)
- User backend process
processus postgres, processus serveurs

Architecture





LA GESTION DES UTILISATEURS

- Chaque cluster de base de données contient un ensemble d'utilisateurs de base de données. Ces utilisateurs sont distincts des utilisateurs gérés par le système d'exploitation sur lequel le serveur s'exécute. Les utilisateurs possèdent des objets de base de données (par exemple, des tables) et peuvent attribuer des privilèges sur ces objets à d'autres utilisateurs pour contrôler qui a accès à quel objet.
- Comment créer et gérer des utilisateurs et introduit le système de privilèges.

Level 1: Turning the TCP on or off (listen_addresses in postgresql.conf, and so on)

Level 2: Network authentication (pg_hba.conf)

Level 3: Instance-level permissions

Level 4: Database-level permissions

Level 5: Schema-level permissions

Level 6: Table-level permissions

Level 7: Column permissions

HBA.CONF

- # TYPE DATABASE USER ADDRESS METHOD
"local" is for Unix domain socket connections only local all all trust

IPv4 local connections:

host all all 127.0.0.1/32 trust

IPv6 local connections

host all all ::1/128 trust

remote access

host all all 192.168.0.0/16 md5

- host all all 213.43.13.0/24 pa

LA GESTION DES UTILISATEURS

- Utilisateurs de la Base de Données
- Les utilisateurs de la base de données sont conceptuellement complètement séparés des utilisateurs du système d'exploitation.
En pratique, il peut être pratique de maintenir une correspondance, mais ce n'est pas obligatoire. Pour créer un utilisateur, utilisez la commande SQL CREATE USER :

`CREATE USER nom`

- Le nom suit les règles pour les identifiants SQL :

soit sans caractères spéciaux,
soit entre guillemets doubles.

Pour supprimer un utilisateur existant, utilisez la commande DROP USER :
`DROP USER nom`

- Pour plus de commodité, les programmes createuser et dropuser sont fournis encapsulées dans des commandes SQL qui peuvent être appelées depuis la ligne de commande du shell :
`createuser nom`
`dropuser nom`

LA GESTION DES UTILISATEURS

- Utilisateurs de la Base de Données
- A l'initialisation d'une base de données un utilisateur prédéfini.
Cet utilisateur aura l'ID fixe 1, et par défaut (sauf si modifié lors de l'exécution de initdb), il aura le même nom que l'utilisateur du système d'exploitation qui a initialisé le cluster de base de données.
En général, cet utilisateur sera nommé postgres.
Pour créer plus d'utilisateurs, vous devez d'abord vous connecter en tant qu'utilisateur initial.
- Une seule identité utilisateur est active pour une connexion au serveur de base de données.
Le nom d'utilisateur à utiliser pour une connexion à une base de données particulière est indiqué par le client qui initie la demande de connexion de manière spécifique à l'application.
Par exemple, le programme psql utilise l'option de ligne de commande -U pour indiquer l'utilisateur à se connecter en tant que.
- L'ensemble des utilisateurs de base de données auxquels une connexion cliente donnée peut se connecter est déterminé par la configuration de l'authentification client.
- Dans un environnement multi utilisateur, il est recommandé de définir les privilèges pour chaque utilisateur créé en base → Gestion des profils pour des accès plus complexes

LA GESTION DES UTILISATEURS

- **Attributs de l'utilisateur**

Un utilisateur de base de données peut avoir un certain nombre d'attributs qui définissent ses privilèges et interagissent avec le système d'authentification client.

- **superutilisateur**

Un superutilisateur de base de données contourne toutes les vérifications de permission.

De plus, seul un superutilisateur peut créer de nouveaux utilisateurs. Pour créer un superutilisateur de base de données, utilisez `CREATE USER nom CREATEUSER`.

- **création de base de données**

Un utilisateur doit explicitement se voir accorder la permission de créer des bases de données (sauf pour les superutilisateurs)

Pour créer un tel utilisateur, utilisez `CREATE USER nom CREATEDB`.

- **mot de passe**

Un mot de passe est nécessaire si la méthode d'authentification client exige que l'utilisateur fournisse un mot de passe lors de la connexion à la base de données.

Les méthodes d'authentification `passwd`, `md5` et `crypt` utilisent des mots de passe.

Les mots de passe de la base de données sont distincts des mots de passe du système d'exploitation.

Spécifiez un mot de passe lors de la création de l'utilisateur avec `CREATE USER nom PASSWORD 'string'`.

LA GESTION DES UTILISATEURS

- **Attributs de l'utilisateur**

Les attributs d'un utilisateur peuvent être modifiés après la création avec ALTER USER.

- Un utilisateur peut également définir des valeurs par défaut personnelles pour de nombreux paramètres de configuration d'exécution. Par exemple, si pour une raison quelconque vous souhaitez désactiver les analyses d'index (indice : pas une bonne idée) à chaque fois que vous vous connectez, vous pouvez utiliser

```
ALTER USER monnom SET enable_indexscan TO off;
```

Cela enregistrera le paramètre (mais ne le définira pas immédiatement) et lors des connexions ultérieures, il semblera que SET enable_indexscan TO off; ait été appelé juste avant le démarrage de la session. Vous pouvez toujours modifier ce paramètre pendant la session ; ce ne sera que le paramètre par défaut. Pour annuler un tel paramètre, utilisez

```
ALTER USER nom_utilisateur RESET nom_variable;
```

LA GESTION DES UTILISATEURS

- **Groupes**

Comme dans Unix, les groupes sont un moyen de regrouper logiquement des utilisateurs pour faciliter la gestion des privilèges : les privilèges peuvent être accordés à un groupe dans son ensemble ou révoqués de celui-ci. Pour créer un groupe, utilisez

```
CREATE GROUP nom
```

Pour ajouter des utilisateurs à un groupe ou en supprimer, utilisez

```
ALTER GROUP nom AJOUTER UTILISATEUR uname1, ...
```

```
ALTER GROUP nom SUPPRIMER UTILISATEUR uname1, ...
```

LA GESTION DES UTILISATEURS

- **Privilèges**

Lorsqu'un objet de base de données est créé, il se voit attribuer un propriétaire. Le propriétaire est l'utilisateur qui a exécuté l'instruction de création. Pour changer le propriétaire d'une table, d'un index, d'une séquence ou d'une vue, utilisez la commande ALTER TABLE. Par défaut, seul le propriétaire (ou un superutilisateur) peut faire quoi que ce soit avec l'objet. Afin de permettre à d'autres utilisateurs de l'utiliser, des privilèges doivent être accordés.

Il existe plusieurs types de privilèges différents :

SELECT, INSERT, UPDATE, DELETE, RULE, REFERENCES, TRIGGER, CREATE, TEMPORARY, EXECUTE, USAGE et ALL PRIVILEGES.

Pour plus d'informations sur les différents types de privilèges pris en charge par PostgreSQL, consultez la page GRANT dans le Manuel de Référence PostgreSQL.

Le droit de modifier ou de détruire un objet est toujours le privilège du propriétaire uniquement.

Pour attribuer des privilèges, la commande GRANT est utilisée. Ainsi, si joe est un utilisateur existant et accounts est une table existante, le privilège de mettre à jour la table peut être accordé avec GRANT UPDATE ON accounts TO joe;

L'utilisateur exécutant cette commande doit être le propriétaire de la table.

Pour accorder un privilège à un groupe, utilisez GRANT SELECT ON accounts TO GROUP staff;

L'utilisateur spécial "PUBLIC" peut être utilisé pour accorder un privilège n'importe quel utilisateur du système.

Écrire ALL à la place d'un privilège spécifique spécifie que tous les privilèges seront accordés.

LA GESTION DES UTILISATEURS

- **Privilèges**

Pour révoquer un privilège, utilisez la commande REVOKE de manière appropriée :

```
REVOKE ALL ON accounts FROM PUBLIC;
```

Les privilèges spéciaux du propriétaire de la table (c'est-à-dire le droit de faire DROP, GRANT, REVOKE, etc.) sont toujours implicites en tant que propriétaire et ne peuvent pas être accordés ou révoqués.

Mais le propriétaire de la table peut choisir de révoquer ses propres privilèges ordinaires, par exemple pour rendre une table en lecture seule pour lui-même ainsi que pour les autres.

LA GESTION DES UTILISATEURS

- **Fonctions et Triggers**

Les fonctions et les triggers permettent aux utilisateurs d'insérer du code dans le serveur backend que d'autres utilisateurs peuvent exécuter sans le savoir.

Ainsi, les deux mécanismes permettent aux utilisateurs d'inclure des chevaux de Troie chez d'autres avec une relative impunité. La seule véritable protection est un contrôle strict sur qui peut définir des fonctions.

Les fonctions écrites dans n'importe quel langage sauf SQL s'exécutent à l'intérieur du processus du serveur backend avec les permissions du système d'exploitation du processus du démon du serveur de base de données.

Il est possible de modifier les structures de données internes du serveur à partir de trusted functions (fonctions de confiance).

Ainsi, entre autres choses, de telles fonctions peuvent contourner tout contrôle d'accès au système.

C'est un problème inhérent aux fonctions C définies par l'utilisateur.

BACKUP / RESTORE

- **Backup**

Stockage des données importantes et vitals → les bases de données PostgreSQL doivent être régulièrement sauvegardées.

Même si la procédure de sauvegarde sous postgres est simple, il est important d'en avoir une bonne compréhension des techniques sous-jacentes.

Il existe deux approches fondamentalement différentes pour exécuter une sauvegarde des données sous PostgreSQL :

- Sauvegarde logique : SQL dump → Sauvegarde à chaud
- Sauvegarde physique : File system level backup → Sauvegarde à froid.

BACKUP / RESTORE

- **Backup**

- SQL dump

Cette method consiste en la generation d'un fichier texte contenant des commandes sql qui lorsque exécutées dans une base de données, recréera la base de données sauvegardé à l'identique, au moment ou la sauvegarde a été faite.

l'outil pg_dump est un programme fourni par postgres pour cette fonction.

Pg_dump est une application cliente, elle peut être exécutée depuis un serveur distant

```
pg_dump dbname > outfile
```

- File system level backup

BACKUP / RESTORE

- **Backup**

- SQL dump

➔ `pg_dump` n'opère pas avec des autorisations spéciales.

En particulier, vous devez avoir un accès en lecture à toutes les tables que vous souhaitez sauvegarder, donc en pratique, vous devez presque toujours être un superutilisateur de la base de données.

Pour spécifier quel serveur de base de données `pg_dump` doit contacter, utilisez les options en ligne de commande `-h host` et `-p port`.

L'hôte par défaut est l'hôte local ou ce que spécifie votre variable d'environnement `PGHOST`.

De même, le port par défaut est indiqué par la variable d'environnement `PGPORT`

Comme toute autre application cliente PostgreSQL, `pg_dump` se connectera par défaut avec le nom d'utilisateur de la base de données qui est égal au nom d'utilisateur du système d'exploitation actuel.

Pour outrepasser cela, spécifiez soit l'option `-U`, soit définissez la variable d'environnement `PGUSER`.

BACKUP / RESTORE

- **Backup**

- SQL dump

Les sauvegardes créées par `pg_dump` sont cohérentes et consistente, c'est-à-dire que les mises à jour de la base de données pendant que `pg_dump` s'exécute ne seront pas dans la sauvegarde.

`pg_dump` ne bloque pas les autres opérations sur la base de données pendant qu'il travaille. (Les exceptions sont les opérations qui nécessitent de fonctionner avec un verrouillage exclusif, comme `VACUUM FULL`.)

Important : Lorsque votre schéma de base de données repose sur les OID (par exemple comme clés étrangères), vous devez instruire `pg_dump` pour sauvegarder également les OID. Pour ce faire, utilisez l'option en ligne de commande `-o`.

Les « Large objets » ne sont pas sauvegardés par défaut non plus.

BACKUP / RESTORE

- **Restore**

- `psql`

Les fichiers générés par `pg_dump` sont utilisés dans la commande `psql` pour restaurer une base de données postgresql.

`Psql dbname < infile`

Le fichier `infile` est celle générée dans la commande `pg_dump`.

La base de données doit être créée au préalable, avant de procéder à la restauration de la base de données.

`Pg_dump` et `psql` ont la possibilité de lire ou d'écrire dans un pipe nommé `(|)`

`pg_dump -h hosts1 dbname | psql -h host2 dbname.`

BACKUP / RESTORE

- **Restore**

- `pg_dumpall`

Pour faire un full backup de votre cluster de base de données, vous devez utiliser la commande `pg_dumpall`.

Cette commande sauvegardera toutes les bases de données d'un cluster contrairement à `pg_dump` qui ne sauvegardera qu'une base de données

```
pg_dump_all > outfile
```

REPLICATION

Ce chapitre est dédié à tout ce qui concerne le journal des transactions PostgreSQL ainsi que la réplication.

Les sujets clés suivants seront abordés dans ce chapitre :

- Comprendre le journal des transactions PostgreSQL
- Configuration des points de contrôle
- Configuration d'une réplication asynchrone Mise à niveau vers une réplication synchrone

REPLICATION

- Comprendre PostgreSQL journal des transactions

Avant de plonger dans le monde de la réplication, il est logique de s'arrêter un moment et de se concentrer notre attention sur le journal des transactions PostgreSQL <> **Transaction log**.

Quels sont les principes de base derrière cette structure de données très importante qui est capable de rendre PostgreSQL aussi robuste qu'il l'est

REPLICATION

- **Le but du journal des transactions**

En théorie, le but du journal des transactions PostgreSQL ou Write Ahead Log (WAL) est assez simple.

- Il garantit qu'une instance de base de données puisse survivre aux pannes.

En cas de crash du système d'exploitation, le journal des transactions permettra de réparer le système et aider les administrateurs système à éviter un désastre.

Alors, pourquoi une instance de base de données a-t-elle besoin d'une réparation en cas de panne ?

Que se passe t-il lorsque nous avons une table, (environ 10 Go) et nous souhaitons ajouter une ligne simple, comme indiqué dans le code suivant :

```
INSERT INTO mytable VALUES ('abcdefg');
```

REPLICATION

- **Le but du journal des transactions**

PostgreSQL va rechercher sur le disque un ensemble de bloc contiguë libre, contenant suffisamment d'espace pour stocker la ligne.

Si un tel espace est disponible, PostgreSQL le récupère et commence les écritures de la ligne a,b,c ...

Supposons que le système plante pendant l'écriture ...

Lorsque le système redémarre, le problème est que nous avons une ligne inachevée et interrompue quelque part dans le tableau.

Nous ne pourrions peut-être même pas le localiser facilement et efficacement.

Le journal des transactions (xlog ou WAL) est là pour résoudre exactement ce genre de problème.

Comment ça marche?

Le xlog peut être vu comme une bande de changements binaires.

REPLICATION

Conseil

Gardez à l'esprit que le journal des transactions ne contient pas l'instruction SQL : il contient en réalité un ensemble de modifications binaires.

Rejouer SQL ne conduirait pas nécessairement aux mêmes résultats. Toutefois, les changements binaires sont sûrs et efficaces.

Lorsque le système tombe en panne, les modifications peuvent être prises et rejouées pour réparer les fichiers de données, etc.

Mais pourquoi l'écriture dans le journal des transactions est-elle plus sûre que l'écriture directe dans la table ?

La raison est la suivante :

- il s'agit d'une écriture séquentielle : l'ordre des changements est toujours clair à 100 %.

En plus de cela, un utilisateur final ne recevra un message COMMIT réussi que si nous l'avons enregistré en toute sécurité dans le journal des transactions. Si le système échoue déjà lors d'une écriture dans le journal des transactions, l'utilisateur final ne recevra de toute façon pas de message COMMIT. En replay, on peut voir la fin du journal des transactions et s'arrêter après la dernière entrée valide.

Bien entendu, il ne s'agit que d'une brève introduction au sujet qui ne couvre pas tous les détails techniques de ce domaine ;

REPLICATION

- Inspection de la taille du journal des transactions

Une fois que vous avez compris l'objectif fondamental du transaction logs (journal des transactions), comment les choses sont réellement organisées sur le disque.

Le journal des transactions est (par défaut) stocké dans un répertoire appelé `pg_xlog` situé dans le dossier data
répertoire (`$PGDATA`).

PostgreSQL organise le journal des transactions en segments de 16 Mo.

REPLICATION

- Inspection de la taille du journal des transactions

```
[hs@ferdi pg_xlog]$ ls -l
```

```
total 131072
```

```
-rw----- 1 hs hs 16777216 Jul 23 13:31 000000010000000500000075
-rw----- 1 hs hs 16777216 Jul 23 09:10 000000010000000500000076
-rw----- 1 hs hs 16777216 Jul 23 09:10 000000010000000500000077
-rw----- 1 hs hs 16777216 Jul 23 09:10 000000010000000500000078
-rw----- 1 hs hs 16777216 Jul 23 09:10 000000010000000500000079
-rw----- 1 hs hs 16777216 Jul 23 09:10 00000001000000050000007A
-rw----- 1 hs hs 16777216 Jul 23 09:10 00000001000000050000007B
-rw----- 1 hs hs 16777216 Jul 23 09:10 00000001000000050000007C
```


REPLICATION

Ce qui est intéressant ici, c'est le nombre de fichiers :
dans PostgreSQL, le nombre de fichiers journaux de transactions est constant. Ainsi, même si vous exécutez une transaction très longue dépassant largement la taille du journal des transactions, le nombre de fichiers n'augmentera pas.

Voici les deux formules utilisées pour calculer la taille du journal des transactions dans PostgreSQL :

$$(2 + \text{checkpoint_completion_target}) * \text{checkpoint_segments} + 1$$

The second formula is as follows:

$$\text{checkpoint_segments} + \text{wal_keep_segments} + 1$$

REPLICATION

- **statut_archive**

La signification de ces paramètres de configuration sera abordée plus loin dans ce chapitre.

Cependant, il est important de comprendre que la manière dont la base de données est utilisée par l'application n'affecte pas la taille du xlog ; cependant, la configuration de la base de données est affectée.

Comment les fichiers xlog sont-ils organisés ? Si vous examinez l'un de ces fichiers, vous verrez qu'ils sont fondamentalement illisibles. Il est impossible de modifier les fichiers xlog

REPLICATION

- **Checkpoint**

Supposons que nous insérons des données comme nous l'avons fait précédemment :

```
INSERT INTO mytable VALUES ('abcdefg');
```

Le but de cette instruction INSERT est d'écrire les nouvelles données dans une table.

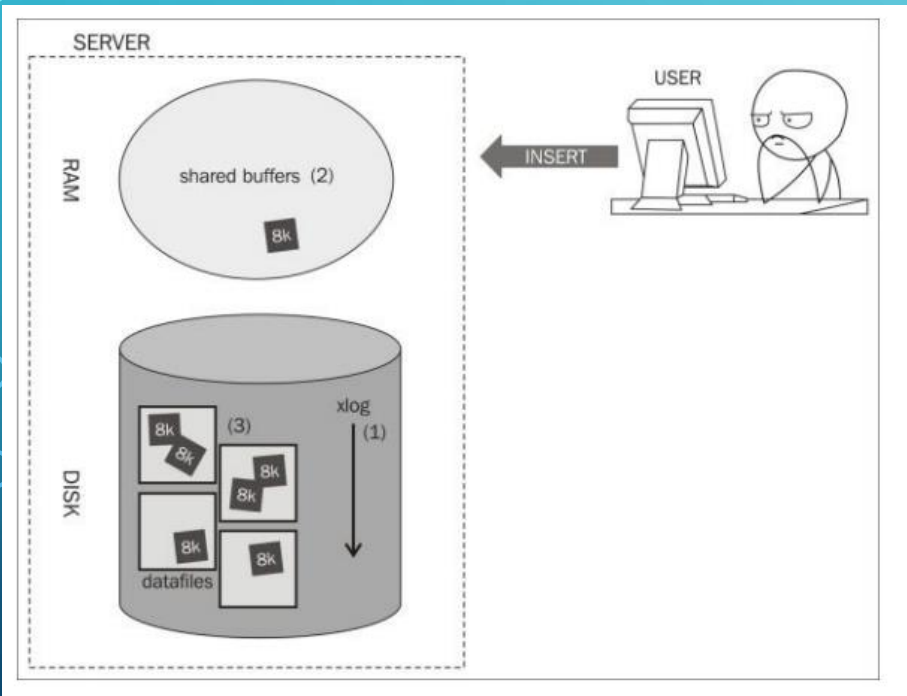
Dans PostgreSQL, une table est toujours représentée par quelques fichiers de données constitués de 8 000 blocs (sauf modification au moment de la compilation).

Question : est-il souhaitable de écrire directement dans la table ?

Comme indiqué plus haut dans ce chapitre, écrire directement dans la table Ce n'est de toute façon pas réalisable car en cas d'accident, nous risquons de perdre les données.

REPLICATION

- Checkpoint



Une fois les données envoyées au journal des transactions, une copie du futur bloc sur le disque sera placée dans les tampons partagés de PostgreSQL :

Pourquoi ces précautions préalables ?

→ en cas de crash, vous pouvez toujours réparer la table en vous basant sur le xlog.

Est-il nécessaire d'écrire rapidement des données dans la table de données ? Eh bien pas vraiment.

En retardant l'écriture sur la table de données sous-jacente, de nombreuses modifications peuvent être écrites sur le disque en une seule E/S disque.

Optimisation des E/S → Gain de temps.

→ Gestion de la consistance de la donnée

REPLICATION

- **Checkpoint**

Problème : Pouvons nous écrire indéfiniment dans le fichier xlog sans jamais écrire dans les fichiers de données sous-jacents.

N'oubliez pas que le fichier xlog a une taille fixe.

Réponse : Checkpoint ou point de contrôle.

L'idée du checkpoint est de s'assurer que l'écriture définitive sur disque est réalisé à intervalle régulier.

➔ Le réglage des checkpoints (points de contrôle) peut avoir un impact significatif sur les performances

REPLICATION

- **Checkpoint**

➔ Nécessité d'optimiser les checkpoints pour un fonctionnement optimal de Postgres

Intervalle entre les checkpoints

Un des paramètres les plus importants que l'administrateur doit tuner
plus cet intervalle est grand, plus le moteur de base de données peut gérer les tâches de reorganisation,
fusion des écritures etc ... donc accélérer ses écritures

Inconvénient d'avoir un intervalle important entre deux checkpoints est d'avoir un temps de restauration important.

Le temps de redémarrage de la base de données peut être impacté.

Deux variables pour contrôler ce paramètre :

`checkpoint_segments = 3` ➔ 3 segments ($3 * 16$ MB)

`checkpoint_timeout = 5min`

➔ Default sur les bases transactionnelle : 256 segments et 30 minutes.

REPLICATION

- **Réplication Synchrone**

La réplication synchrone : fonctionnalité avancée permettant de garantir que les données sont répliquées sur plusieurs serveurs avant que la transaction soit considérée comme terminée.

Cette approche assure une haute disponibilité et une fiabilité des données, même en cas de défaillance de l'un des serveurs.

MASTER # Chemin du fichier : /etc/postgresql/<version>/main/postgresql.conf

Activer la réplication

wal_level = replica

archive_mode = on

archive_command = 'cp %p /path/to/archive/%f'

max_wal_senders = 5

synchronous_standby_names = 'standby1'

SLAVE # Chemin du fichier : /etc/postgresql/<version>/main/pg_hba.conf

host replication all <standby_server_ip>/32 md5

REPLICATION

- **Réplication Synchrone**

La réplication synchrone dans PostgreSQL assure une haute disponibilité et une récupération rapide en cas de panne. Cependant, il est important de noter que cela peut impacter les performances, car les transactions doivent être confirmées par tous les serveurs synchrones avant d'être validées.

En ajustant les configurations et en testant soigneusement, vous pouvez optimiser l'équilibre entre la performance et la résilience.

REPLICATION

- **Réplication Asynchrone**

La réplication asynchrone sous PostgreSQL est une méthode de réplication où les transactions confirmées sur le serveur principal (master) ne sont pas immédiatement confirmées sur le serveur de secours (standby). Cela permet d'améliorer les performances, car le serveur principal n'a pas besoin d'attendre que les serveurs de secours aient écrit les données avant de confirmer les transactions. Cependant, il peut y avoir un léger délai (lag) entre le serveur principal et les serveurs de secours.

REPLICATION

- **Réplication Asynchrone**

- ➔ Méthode de réplication où les transactions confirmées sur le serveur principal (master) ne sont pas immédiatement confirmées sur le serveur de secours (standby).
- ➔ Cela permet d'améliorer les performances, car le serveur principal n'a pas besoin d'attendre que les serveurs de secours aient écrit les données avant de confirmer les transactions.
- ➔ Cependant, il peut y avoir un léger délai (lag) entre le serveur principal et les serveurs de secours.

Vérification : `SELECT * FROM pg_stat_replication;`

La colonne `sync_state` doit être valorisée à `async`

FÉDÉRATION

La fédération de bases de données sous PostgreSQL permet de distribuer les données entre plusieurs serveurs PostgreSQL, offrant ainsi une solution pour la répartition de la charge, l'augmentation de la disponibilité et l'amélioration des performances.

La fédération permet aux bases de données réparties d'être traitées comme une seule entité logique.

PostgreSQL ne supporte pas nativement la fédération comme certaines autres bases de données, mais il existe plusieurs outils et extensions pour atteindre cet objectif.

L'une des solutions les plus courantes pour la fédération sous PostgreSQL est l'utilisation de l'extension `postgres_fdw` (Foreign Data Wrapper).

FÉDÉRATION

• Cas d'utilisation et avantages

Répartition de la charge : Distribuer les requêtes entre plusieurs serveurs pour améliorer les performances.

Disponibilité élevée : Répliquer les données sur plusieurs serveurs pour assurer la continuité de service.

Intégration de données : Accéder à des données situées sur différents serveurs de manière transparente.

• Limites et considérations

Performance : Les requêtes sur les données distantes peuvent être plus lentes en raison de la latence réseau.

Complexité de la gestion : La configuration et la gestion des serveurs fédérés peuvent être plus complexes.

Sécurité : Assurez-vous que les connexions entre les serveurs sont sécurisées.

INTERNATIONALISATION (i18n)

- L'internationalisation (i18n) sous PostgreSQL concerne la prise en charge des langues, des formats de date et de nombre, ainsi que des jeux de caractères différents. PostgreSQL fournit plusieurs fonctionnalités intégrées pour gérer les besoins d'internationalisation.

- **Principales fonctionnalités pour l'internationalisation sous PostgreSQL**

Encodage des caractères :

PostgreSQL supporte plusieurs encodages de caractères.

Le plus couramment utilisé pour l'internationalisation est UTF-8, qui permet de stocker des caractères de presque toutes les langues du monde.

Lors de la création d'une base de données, spécifiez l'encodage avec **ENCODING**.

```
CREATE DATABASE mydb ENCODING 'UTF8';
```

INTERNATIONALISATION (I18N)

- Collations :

Les collations déterminent la manière dont les chaînes de caractères sont comparées et triées. PostgreSQL permet de définir des collations spécifiques pour des langues ou des régions particulières. Vous pouvez spécifier la collation lors de la création d'une base de données ou d'une colonne.

```
CREATE DATABASE mydb LC_COLLATE 'en_US.UTF-8' LC_CTYPE 'en_US.UTF-8';
```

- Formats de date et heure :

PostgreSQL permet de formater les dates et heures selon les paramètres régionaux grâce à la fonction **TO_CHAR**.

```
SELECT TO_CHAR(NOW(), 'DD Mon YYYY HH24:MI:SS');
```

INTERNATIONALISATION (I18N)

- Localisation des messages d'erreur :

PostgreSQL peut afficher les messages d'erreur dans différentes langues.

Vous pouvez configurer cela en ajustant le paramètre `lc_messages` dans `postgresql.conf`.

```
lc_messages = 'fr_FR.UTF-8'
```

- Support des types de données spécifiques :

PostgreSQL fournit des types de données comme 'money' pour la gestion des devises, qui peuvent être configurés selon les formats régionaux.

INTERNATIONALISATION (I18N)

- Utilisation des extensions pour l'internationalisation

Il existe des extensions PostgreSQL qui facilitent certaines tâches d'internationalisation :

pg_trgm : Pour les recherches de similarité et de trigrammes, utiles dans les recherches de texte où l'orthographe peut varier entre les langues.

unaccent : Pour supprimer les accents des chaînes de caractères, ce qui peut être utile pour normaliser les recherches textuelles.

L'internationalisation sous PostgreSQL est facilitée par un ensemble complet de fonctionnalités intégrées, allant du support des encodages de caractères et des collations à la gestion des formats régionaux et des langues multiples. En utilisant ces fonctionnalités, vous pouvez créer des applications capables de gérer efficacement les exigences linguistiques et culturelles de vos utilisateurs à travers le monde.